CMPE452 – Advanced Deep Learning Techniques and Applications

Part A Report: CNN-Based Image Classification

Intel Image Classification Dataset

Emirhan KARAÇOBAN

1. Data Preparation

The Intel image classification dataset used for this course assignment is sourced from Kaggle.

https://www.kaggle.com/datasets/puneet6060/intel-image-classification

The dataset used for the image classification task of this course assignment contains a total of 6 categories, including buildings, forests and other scene types. The training set has around 14,000 images, and the test set has 3,000 images; all are JPEG-format RGB images with a size of 150×150 pixels.

The computer vision dataset for this assignment is hosted on Kaggle. The dataset is first divided into a full training set and a test set. The training set is then split at an 11,228 (%80) training images, 2,806 (%20) validation images, and 3,000 test images.

Section	Number of Images
Train	11.228
Validation	2.806
Test	3.000
Total	17.034

This assignment makes public the full image preprocessing workflow: first, all images are resized to 64×64 pixels, then per-channel normalization is performed using a mean value of 0.5 and a standard deviation of 0.5. To increase data diversity in the training set, I applied data enhancement techniques such as RandomHorizontalFlip, 15-degree RandomRotation. I only applied rescaling and normalization to the test and validation sets.

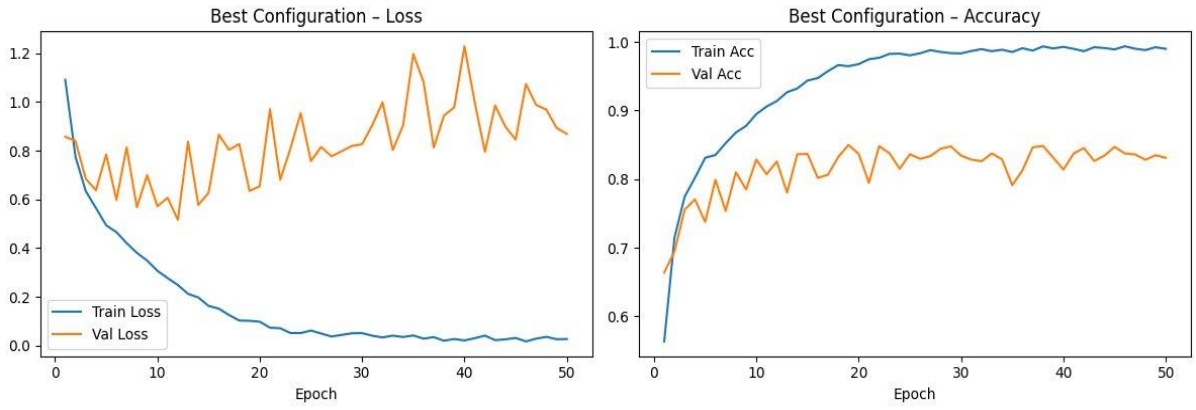
Below are sample images from the training set:

2. Model Architecture and Hyperparameter Choices

In this course assignment, I developed a CNN model based on PyTorch. This assignment splits the self-developed classification deep learning model into two core modules: convolutional blocks and fully connected layers. It sorts out the construction sequence of the network components, and sets 6 neurons in the output layer, each of which corresponds to one classification category.

Hiperparametre	Baseline Value
num_conv_layers	3
conv_kernel_size	3
pool_kernel_size	2
pool_stride	2
conv_dropout	0.2
fc_hidden_units	256
fc_dropout	0.5
activation_fn	relu
optimizer_type	adam
batch_size	32
num_epochs	10

All hyperparameters of this baseline model are listed as follows.



The baseline model of this experiment achieved a validation accuracy of 79.15%.

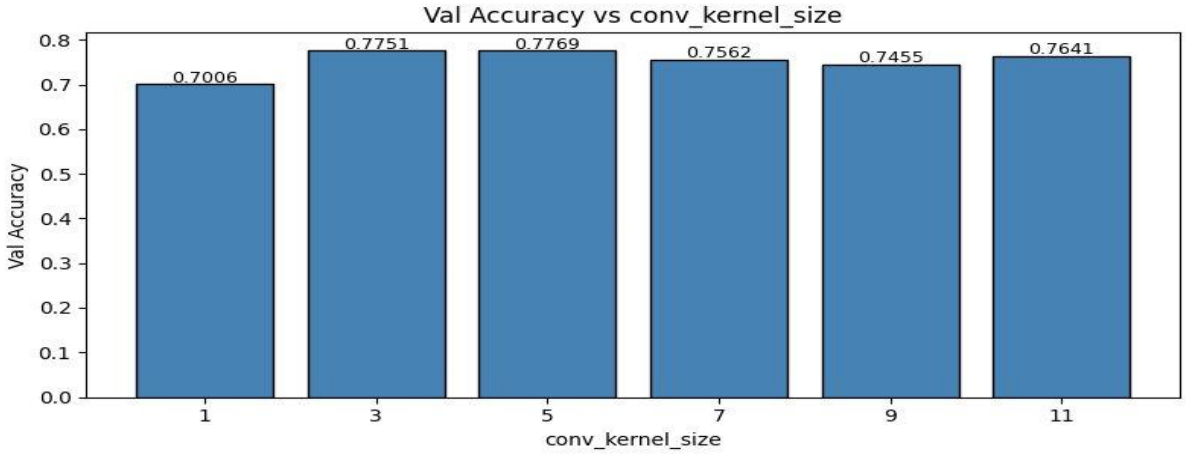
3. Training and Experimentation

This assignment adopts a controlled experimental method: only one hyperparameter is adjusted per round, while all other parameters are maintained at their baseline values. The following text will sequentially present the experimental results for each hyperparameter along with their corresponding interpretations.

3.1 conv_kernel_size

Test values used in this study: 1, 3, 5, 7, 9, 11. I obtained the highest validation accuracy at 5 with 0.7769. Small kernel sizes (1) cannot extract sufficient features, while large kernel sizes (9, 11) become more susceptible to noise and reduce performance. In this assignment, the 5×5 convolution kernel, when adapted to the dataset used in this research, can capture the optimal mid-range features.

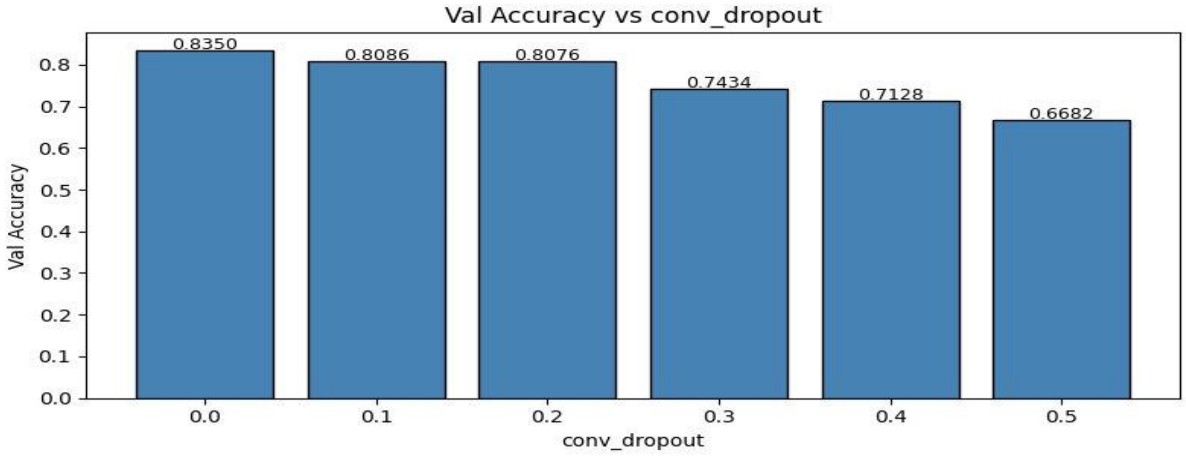
conv_kernel_size	Val Accuracy
1	0.7006
3	0.7751
5 (Best)	0.7769
7	0.7562
9	0.7455
11	0.7641



3.2 conv_dropout

This assignment conducted parameter tuning tests for the convolutional layer dropout rate, targeting the dataset used in this research. A total of 6 groups of parameters ranging from 0.0 to 0.5 were set. It was observed that when the dropout rate was set to 0.0, the validation accuracy reached a peak of 0.8350, and the accuracy kept declining continuously as the parameter value increased. It is inferred that applying dropout to the convolutional layer causes information loss.

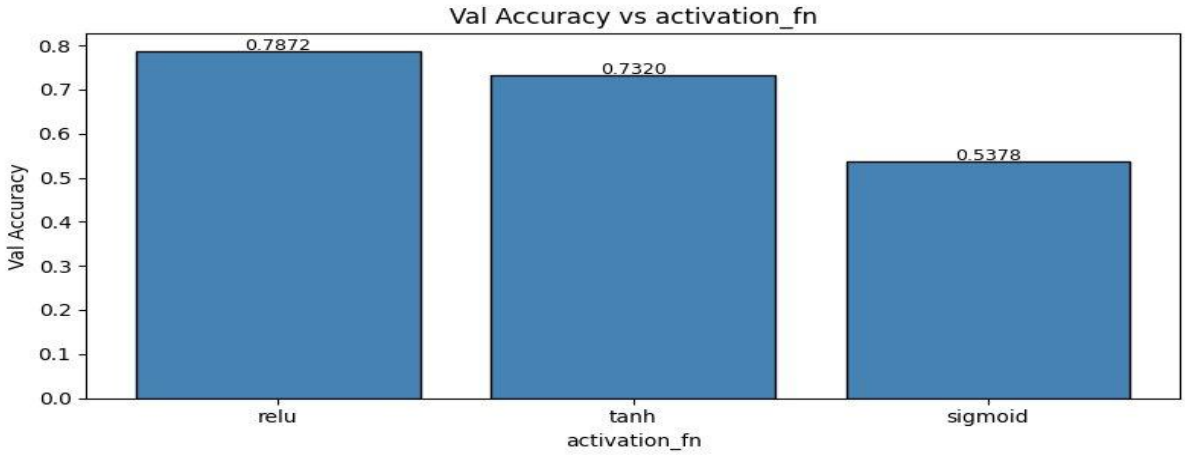
conv_dropout	Val Accuracy
0.0 (Best)	0.8350
0.1	0.8086
0.2	0.8076
0.3	0.7434
0.4	0.7128
0.5	0.6682



3.3 activation_fn

This assignment tested three types of activation functions, and ReLU achieved the highest validation accuracy. In the tests conducted on the deep neural network built for this course assignment, the Sigmoid function produced an output of 0.5378, which verified the existence of the vanishing gradient problem, while the Tanh function only achieved moderate performance.

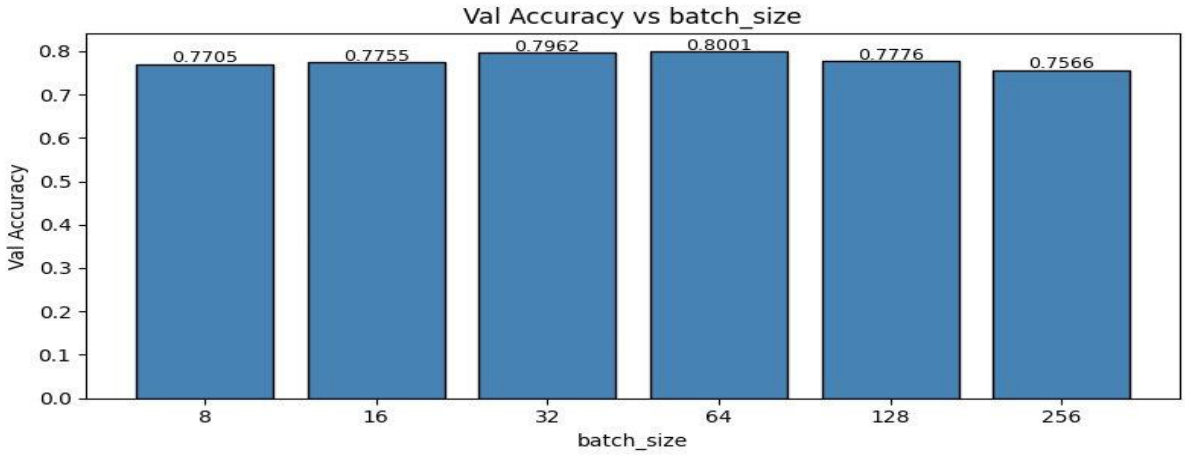
activation_fn	Val Accuracy
relu (Best)	0.7872
tanh	0.7320
sigmoid	0.5378



3.4 batch_size

This test conducted a single parameter test, and the highest validation accuracy was obtained when the value 64 was selected from six groups of candidate values. All values gave very similar results. Very small batch sizes (8) lead to noisy gradients, while very large batch sizes (256) reduce the generalization capacity.

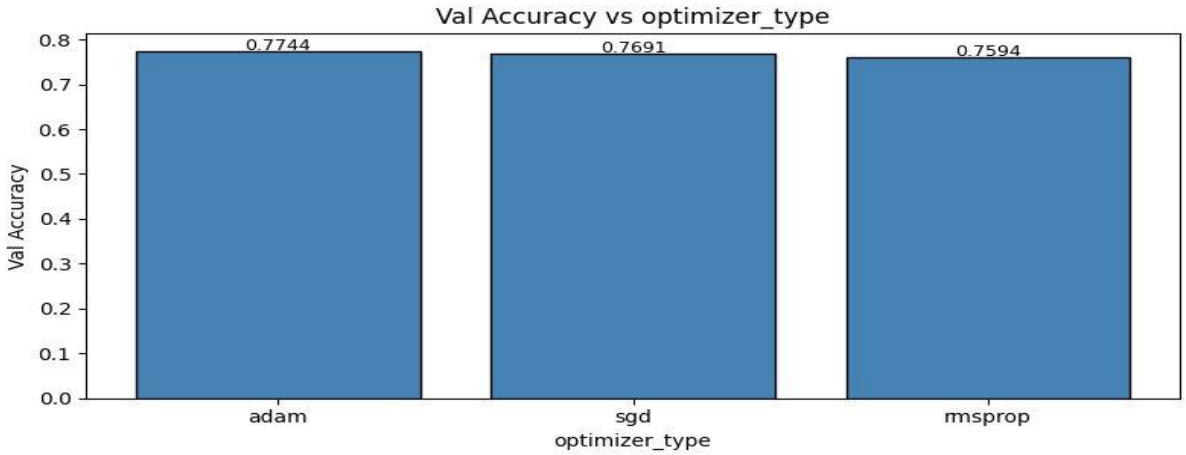
batch_size	Val Accuracy
8	0.7705
16	0.7755
32	0.7962
64 (Best)	0.8001
128	0.7776
256	0.7566



3.5 optimizer_type

This part tests three optimizers: Adam, SGD, and RMSprop. I achieved the highest validation accuracy with Adam at 0.7744. Adam's adaptive learning rate mechanism provided the best convergence performance on this dataset. The performance of SGD approaches the baseline, while RMSprop delivers the worst performance.

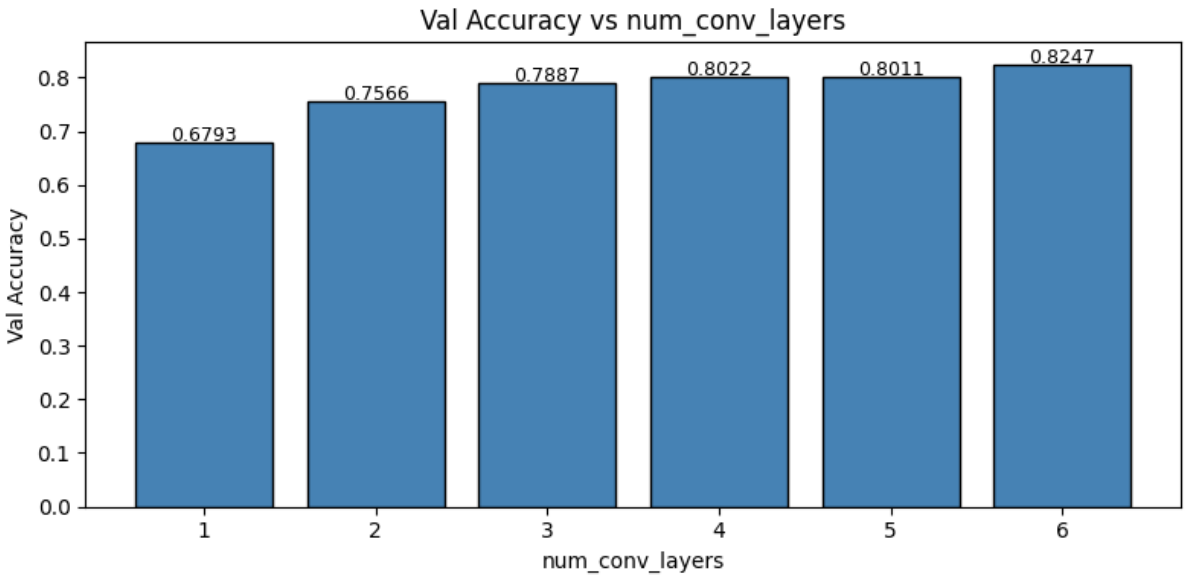
optimizer_type	Val Accuracy
adam (Best)	0.7744
sgd	0.7691
rmsprop	0.7594



3.6 num_conv_layers

This part tested all models with 1 to 6 layers, and the 6-layer model achieved a validation accuracy of 0.8264. I observed that the validation accuracy increased regularly as the number of layers increased. Deep networks can learn more abstract discriminative features.

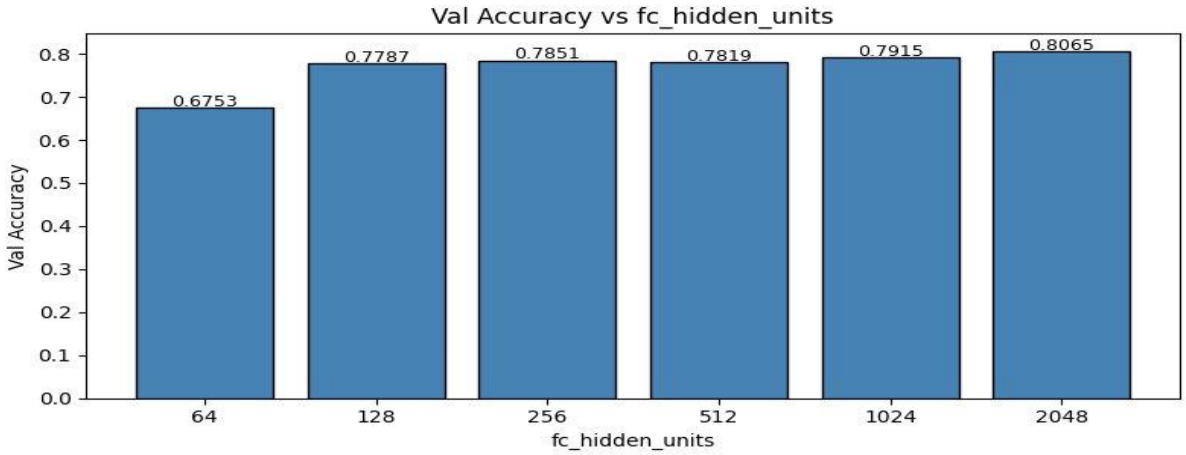
num_conv_layers	Val Accuracy
1	0.6842
2	0.7559
3	0.7783
4	0.7858
5	0.7954
6 (Best)	0.8264



3.7 fc_hidden_units

I tested the values 64, 128, 256, 512, 1024, and 2048. I obtained the highest validation accuracy at 2048, with a value of 0.8065. As the number of neurons increased, the model's learning capacity increased, and performance improved.

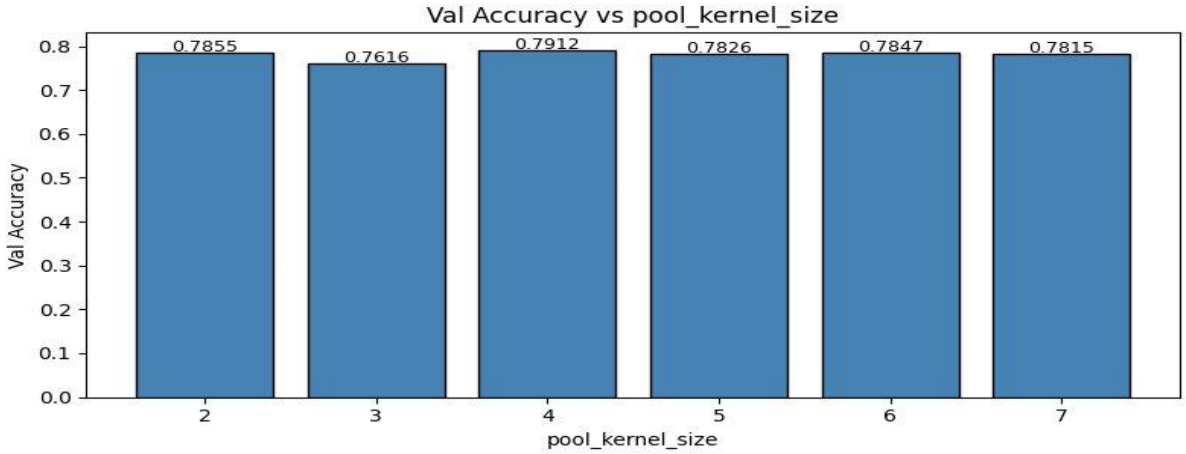
fc_hidden_units	Val Accuracy
64	0.6753
128	0.7787
256	0.7851
512	0.7819
1024	0.7915
2048 (Best)	0.8065



3.8 pool_kernel_size

This part conducted an ablation experiment testing six groups of pooling kernels with sizes of 2, 3, 4, 5, 6, and 7. The 4×4 pooling kernel achieved the optimal performance in preserving spatial information, with a validation accuracy of 0.7912.

pool_kernel_size	Val Accuracy
2	0.7855
3	0.7616
4 (Best)	0.7912
5	0.7826
6	0.7847
7	0.7815

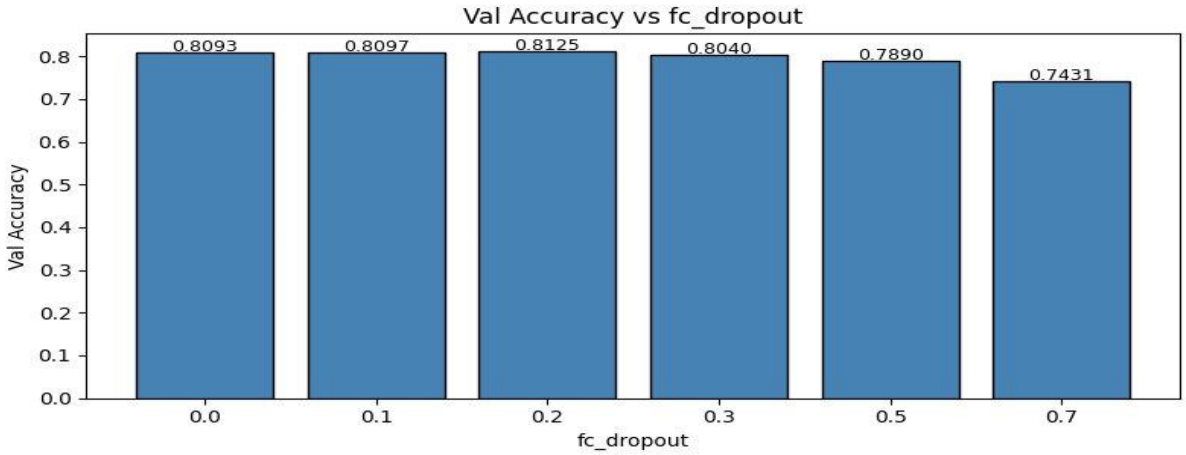


3.9 pool_stride

This study conducted ablation tests on the model's stride parameter. Among the six parameter values tested, the highest validation accuracy of 0.7858 was achieved when the stride was set to 2, while the accuracy only reached 0.1686 when the stride was set to 1. When the stride value was 1, the model only achieved a validation accuracy of 0.1686. In convolutional neural networks, pooling operations with a stride of 1 fail to reduce dimensionality, which will lead to excessively large feature map sizes.

pool_stride	Val Accuracy
1	0.1686
2 (Best)	0.7858
3	0.7659
4	0.7295
5	0.7787

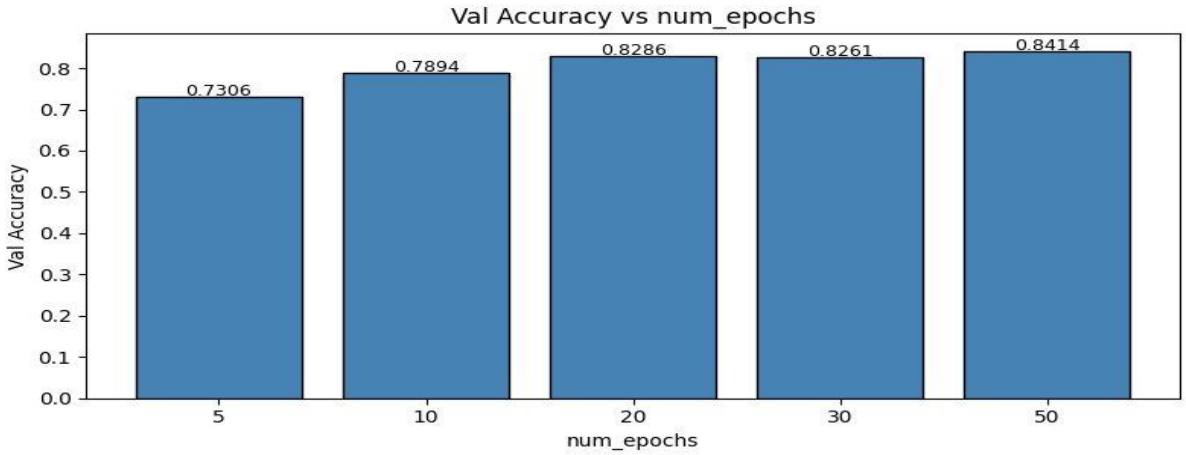
6	0.7156



3.10 fc_dropout

I conducted tuning experiments on deep learning models, testing six groups of dropout values. The optimal value was found to be 0.2, which corresponded to a validation accuracy of 0.8125 and could produce a slight regularization effect.

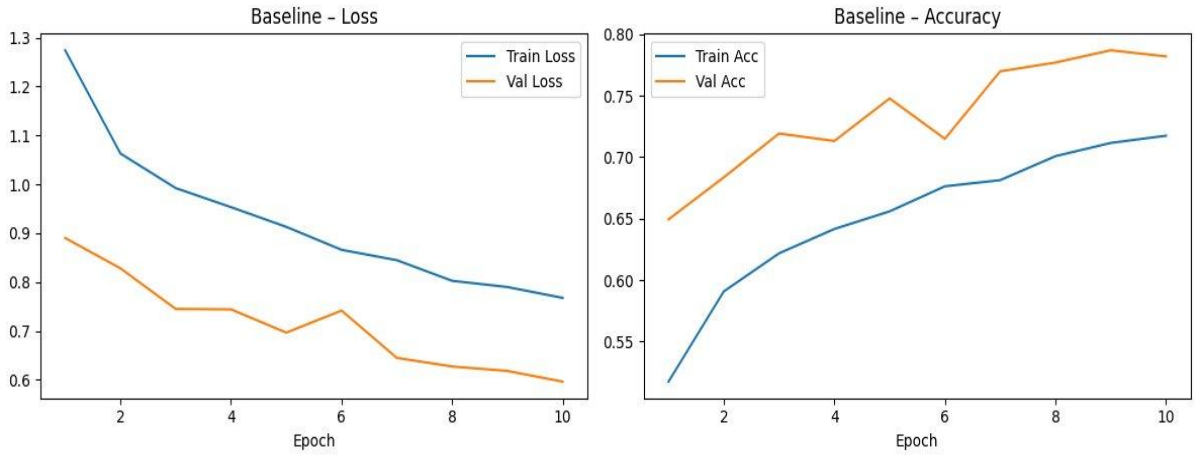
fc_dropout	Val Accuracy
0.0	0.8093
0.1	0.8097
0.2 (Best)	0.8125
0.3	0.8040
0.5	0.7890
0.7	0.7431



3.11 num_epochs

This part tested training epoch counts of 5, 10, 20, 30, and 50. The validation accuracy reached 0.8414 at 50 epochs. Increasing the number of epochs provides more learning opportunities, driving steady improvements in model performance.

num_epochs	Val Accuracy
5	0.7306
10	0.7894
20	0.8286
30	0.8261
50 (Best)	0.8414



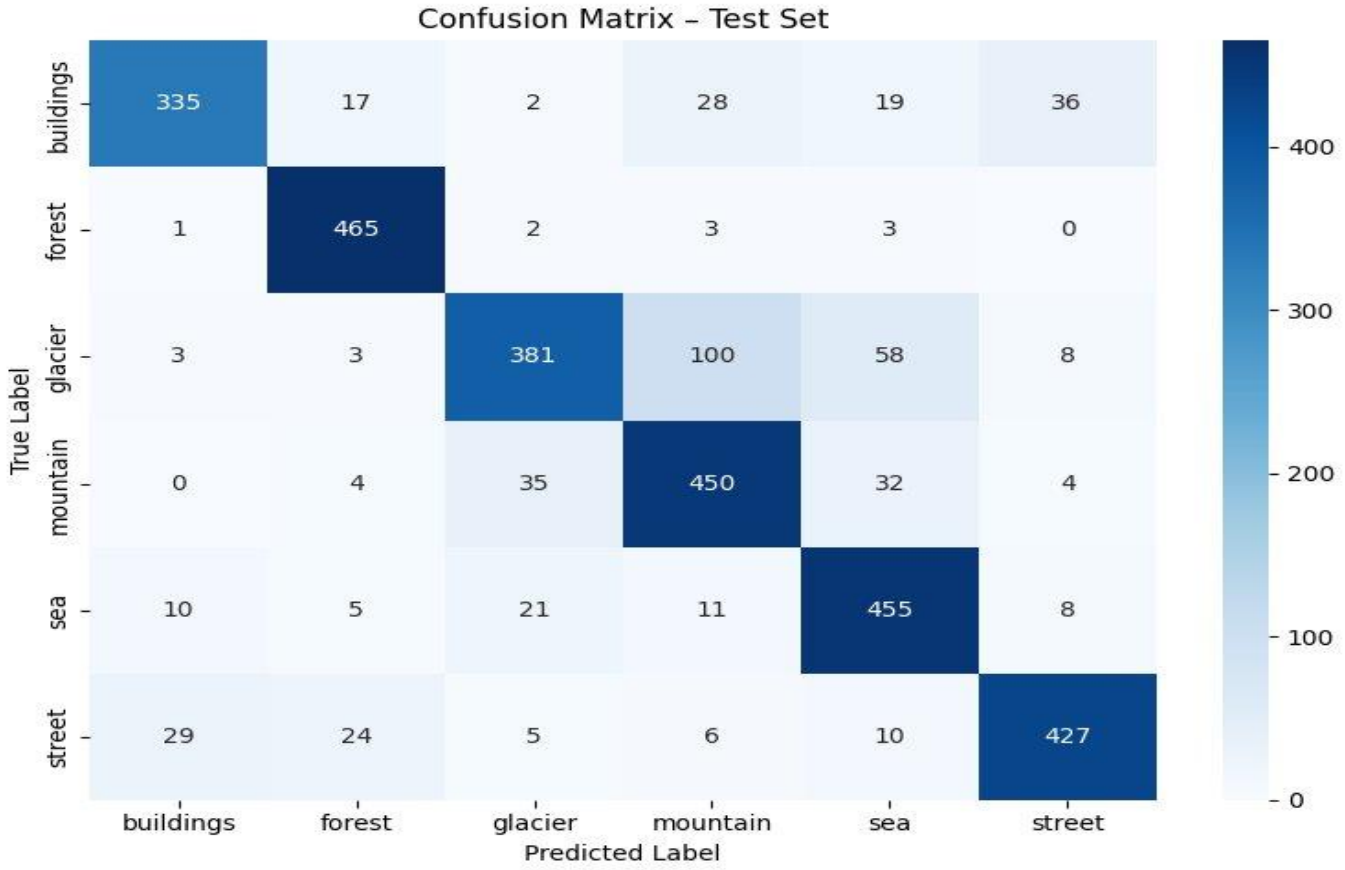
4. Final Evaluation

The best values obtained from all experiments were combined to create the following best configuration:

Hiperparametre	Best Value
conv_kernel_size	5
conv_dropout	0.0
activation_fn	relu
batch_size	64
optimizer_type	adam
num_conv_layers	6
fc_hidden_units	2048
pool_kernel_size	2
pool_stride	2

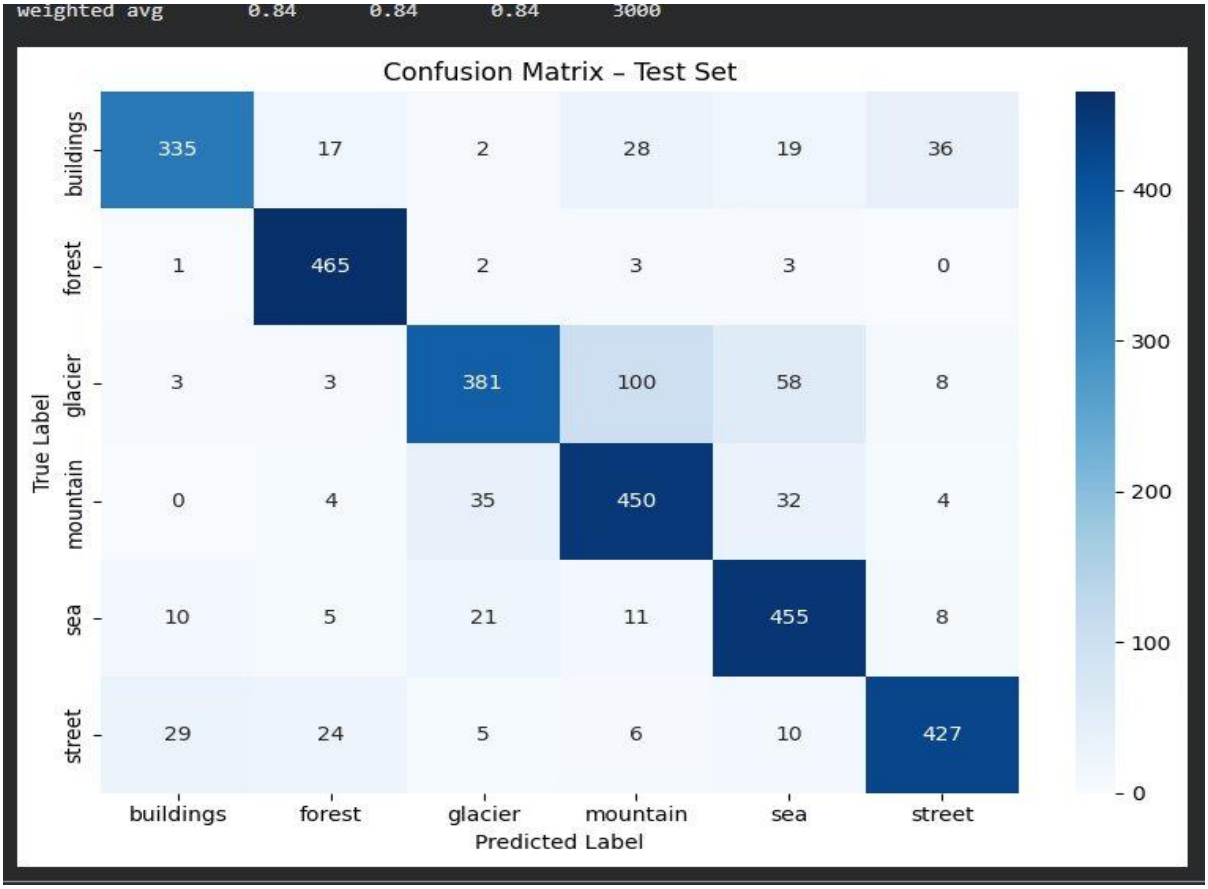
fc_dropout	0.2
num_epochs	50

This study will present the two types of training and validation charts for the model trainedwith the specified configuration used in this research in the following sections.



The machine learning model with the specific configuration trained in this study achieved 84% accuracy on the test set, and its weighted average precision, recall, and F1-score all equaled 0.84.

Confusion matrix analysis shows that this model achieves the highest recognition accuracy for forests and oceans. Confusion was observed between the glacier and mountain classes. This is due to both classes containing similar nature photographs. A similar confusion was observed between the buildings and street classes, because both classes contain images of urban environments.



5. Conclusion

In this assignment, I developed a CNN model using PyTorch on the Intel Image Classification dataset and performed various hyperparameter experiments. The baseline model achieves a validation accuracy of 79.15%, while the optimally configured model reaches a test accuracy of 84%.

In the parameter tuning process for this assignment, the core parameters identified are num_conv_layers and conv_dropout. I observed that deeper networks improved performance, while the use of dropout had a negative impact on this dataset. In the activation function experiment, I clearly saw that sigmoid failed due to the vanishing gradient problem.